Université Paris Cité, Campus Saint-Germain-des-Près
UFR de Mathématiques et Informatique
45, rue des Saints-Pères 75006 Paris

MATHÉMATIQUES ET INFORMATIQUE
Sciences
Université Paris Cité

Algorithmique et Programmation 2

Correction du partiel du 22 mars 2024 — durée: 1h30

Tout document interdit. Les calculatrices et les téléphones portables, même prévus à titre d'horloge, sont également interdits.

MERCI DE BIEN INDIQUER VOTRE GROUPE DE TD SUR VOTRE COPIE

Tous les exercices sont indépendants. On précisera bien la signature de chaque fonction Python proposée. Ces signatures seront notées globalement sur 1 point.

Barème sur 23 points (en comptant le point global pour les signatures)

Exercice 1. **(au total sur 5 points)** Pour chaque ligne (indépendante) d'instructions ci-dessous, donner le contenu de la variable `x` après exécution de la ligne considérée. **Expliquer votre réponse (une réponse correcte sans explication ne sera pas comptabilisée).**

```
1  x = len(str(0b101))
2  x = bin(3)+hex(15)
3  import math; x = int(3.2) + math.floor(-3.2)
4  j = 2; x = (1+2j)**j
5  L = [3,1] + list(range(4)); del L[1:3]; x = L.index(2)
6  x = [1,5,3]*2; y = x; y = sorted(y)
7  x = {0,1,2}&{1,2,3} < {2,3,4}|{3,4,5}
8  S = set([1,3]) - {1}; x = S>{0}
9  z = dict(a=1, b=2, c=3); a = 3; z[a] = 2; x = z['a']
10 D = {5:0, 3:5, 1:3}; x = D[D[D[1]]]
```

(1) `x=1` car `0b101` correspond à l'entier 5 (écrit en base 2), donc `str(0b101)` retourne `'5'`, chaîne de caractères de longueur 1.

(2) `bin(3)` retourne `'0b11'` (écriture de 3 en base 2), `hex(15)` retourne `'0xf'` (écriture de 15 en base 16), donc `x` est la chaîne de caractères `'0b110xf'`.

(3) `int(3.2)` vaut 3 (arrondi par suppression de la partie décimale), `math.floor(-3.2)` vaut -4 (partie entière, i.e. arrondi vers $-\infty$), donc `x=-1`.

(4) `2j` correspond au nombre complexe $2i$, alors que `**j` est une élévation au carré puisque `j=2`, donc `x` vaut le nombre complexe $(1+2i)^2 = -3+4i$, c'est-à-dire `x = -3+4j` en Python.

(5) La première instruction fabrique une liste `L = [3, 1, 0, 1, 2, 3]` par concaténation. Les éléments d'indices 1 (inclus) à 3 (exclu) sont ensuite supprimés de cette liste, donc après le `del` on a `L = [3, 1, 2, 3]`. La dernière instruction cherche l'indice de la première occurence de 2 dans la liste, qui vaut 2, donc `x = 2`.

(6) Après la première instruction, on a `x = [1, 5, 3, 1, 5, 3]`. Ensuite la variable `y` pointe sur la même liste que `x`, mais après `y = sorted(y)` la variable `y` est redéfinie comme une version triée de la liste `x`, donc `x` n'est pas modifiée (contrairement à ce qui se serait passé avec `y.sort()`). On a donc toujours `x = [1, 5, 3, 1, 5, 3]` à la fin.

(7) La variable `x` contient le résultat du test d'inclusion stricte (symbole `<`) entre les ensembles (set) `{0,1,2}&{1,2,3} = {1,2}` (intersection) et `{2,3,4}|{3,4,5} = {2,3,4,5}` (union). Le résultat est `x = False` car l'élément 1 est dans le premier ensemble mais pas dans le deuxième.

(8) La variable `S` est un ensemble (set) qui contient uniquement l'entier 3 (puisqu'on retire l'élément 1). On teste ensuite si `S` contient strictement l'ensemble $\{0\}$, ce qui est faux puisque 0 n'est pas dans `S`, donc `x = False`.

(9) On fabrique un dictionnaire `z` dont les paires clé-valeur sont respectivement `('a',1) ('b',2) ('c',3)`, et on rajoute la paire clé-valeur `(3,2)`. À la fin `x` vaut la valeur associée à la clé `'a'`, qui n'a pas changé, donc on a `x = 1`.

(10) `D` est un dictionnaire, et on calcule de proche en proche `x = D[D[D[1]]] = D[D[3]] = D[5] = 0`.

Exercice 2. **(au total sur 2,5 points)** On appelle une fonction `f` avec la syntaxe `f("", False, z=[])`. Parmi les en-têtes de définition de la fonction `f` considérés ci-dessous, lesquels sont corrects et lesquels conduiront à une erreur lors de la définition de la fonction ou lors de son appel ? **Précisez dans chaque cas soit la cause de l'erreur, soit les valeurs des arguments d'entrée de `f` si l'appel est correct.**

```
1 def f(x, y, z):
2 def f(x="", y=True, z):
3 def f(x, y, z=False, t=[]):
4 def f(a=1, b=2, c=3, z=4):
5 def f(z, x, y=True):
```

(1) correct, `x="", y=False, z=[]`

(2) erreur dans la <u>définition</u> de `f` (`z`, argument obligatoire, vient après `x` et `y`, arguments optionnels)

(3) correct, `x="", y=False, z=[], t=[]`

(4) correct, `a="", b=False, c=3, z=[]`

(5) erreur, l'argument `z` est spécifié deux fois lors de l'appel (une première fois de manière positionnelle, valeur `""`, et une deuxième fois par nom avec la valeur `[]`).

Exercice 3. **(au total sur 2 points)** Soit A une matrice carrée.

(1) **(sur 0,5 point)** Quelle est la complexité, en nombre de multiplications matricielles, du calcul de A^n par l'algorithme d'exponentiation rapide ?

La complexité est $O(\ln n)$ (voir cours).

(2) **(sur 1,5 point)** Expliquer comment calculer A^{19} avec cet algorithme. Combien de multiplications matricielles sont-elles nécessaires ?

On écrit successivement

$$\begin{aligned} A^{19} &= A \times A^{18} \\ A^{18} &= A^9 \times A^9 \\ A^9 &= A \times A^8 \\ A^8 &= A^4 \times A^4 \\ A^4 &= A^2 \times A^2 \\ A^2 &= A \times A \end{aligned}$$

Le calcul effectif se fait en reprenant les lignes dans l'ordre inverse. Il y a au total 6 lignes, donc 6 multiplications matricielles à effectuer pour parvenir au résultat.

Exercice 4. **(au total sur 5 points)** On considère une suite $(u_n)_{n\in\mathbb{N}}$ définie par $u_0 = 1$ et la relation de récurrence

$$\forall n \in \mathbb{N}, \quad u_{n+1} = \begin{cases} n + u_n & \text{si } n \text{ est impair,} \\ 2n + u_{n/2} & \text{si } n \text{ est pair.} \end{cases}$$

(1) **(sur 1 point)** Calculer u_{19}.

Il est maladroit de calculer u_1, u_2 jusqu'à u_{19}, car seuls quelques termes sont utiles. On écrit plutôt

$$\begin{aligned} u_{19} &= 36 + u_9 \\ &= 36 + 16 + u_4 \\ &= 36 + 16 + 3 + u_3 \\ &= 36 + 16 + 3 + 4 + u_1 \\ &= 36 + 16 + 3 + 4 + 0 + u_0 \\ &= 60. \end{aligned}$$

(2) **(sur 2 points)** Écrire une fonction **récursive** `suite1(n)` qui prend en entrée un entier naturel n et renvoie la valeur de u_n.

Comme nous l'avons vu en TD, pour éviter les erreurs d'indices il est sage de commencer par réécrire la récurrence en donnant l'expression de u_n (et non u_{n+1}), c'est-à-dire :

$$\forall n \in \mathbb{N}^*, \quad u_n = \begin{cases} n - 1 + u_{n-1} & \text{si } n-1 \text{ est impair,} \\ 2(n-1) + u_{(n-1)/2} & \text{si } n-1 \text{ est pair.} \end{cases}$$

```python
def suite1(n):
  ''' int -> int
  retourne u(n)
  '''
  if n==0:
    return 1
  if (n-1)%2==0:
    return 2*(n-1) + suite1((n-1)//2)
  return n-1 + suite1(n-1)
```

(3) **(sur 2 points)** Écrire une fonction **non récursive** `suite2(n)` qui prend en entrée un entier naturel n et renvoie la liste $[u_0, u_1, \ldots, u_n]$.

Cette fois-ci, on n'utilise plus la récursivité mais une boucle. À la fin d'un tour de boucle, la liste `L` contient tous les termes de la suite jusqu'à u_k. Attention à bien écrire `L[(k-1)//2]` et non `L[(k-1)/2]`, car la deuxième expression provoquerait une erreur (les indices d'une liste doivent être de type `int`, pas `float`).

```python
def suite2(n):
  ''' int -> list
  retourne la liste [u(0), u(1), ..., u(n)]
  '''
  L = [1]
  for k in range(1, n+1):
    if (k-1)%2==0:
      L.append(2*(k-1)+L[(k-1)//2])
    else:
      L.append(k-1+L[k-1])
  return L
```

Exercice 5. **(au total sur 5,5 points)**

(1) **(sur 2 points)** Écrire une fonction `occurrences(s)` qui prend en entrée une chaîne de caractères `s`, et retourne un dictionnaire `D` qui donne, pour chaque caractère `c` présent dans `s`, la liste des indices des occurrences de `c` dans `s`. Par exemple, si `s = 'abcaaca'`, `occurrences(s)` doit retourner le dictionnaire {`'a':[0,3,4,6], 'b':[1], 'c':[2,5]`}.

On parcourt la chaîne `s` avec un indice `i` (boucle `for`), et pour chaque caractère `c = s[i]` on ajoute l'indice `i` à la liste `D[c]` (si `c` n'est pas encore une clé de `D`, on fait simplement `D[c] = [i]`).

```
def occurrences(s):
  ''' str -> dict
  retourne les indices des occurences des caractères de s
  '''
  D = dict()
  for i in range(len(s)):
    c = s[i]
    if c in D:
      D[c].append(i)
    else:
      D[c] = [i]
  return D
```

(2) **(sur 2 points)** Écrire une fonction génératrice `liste(n)`, qui prend en entrée un entier `n` et délivre successivement les n premiers termes de la suite

$$1, 2, 2, 3, 3, 3, 4, 4, 4, 4, 5, 5, 5, 5, 5, \ldots$$

La difficulté est de gérer indépendamment les changements de chiffres et le nombre de termes à délivrer. Dans le code ci-dessous, on utilise 2 variables: `courant` (le chiffre courant), et `compte` (le nombre d'exemplaires de `courant` déjà délivrés).

```
def liste(n):
  ''' int -> generator
  retourne un générateur de la séquence 1, 2, 2, 3, 3, 3, 4, 4, 4, 4, ...
  '''
  courant = 1 # entier à délivrer
  compte = 0 # nombre de valeurs de courant déja délivrées
  for i in range(n):
    if compte==courant:
      courant += 1
      compte = 0
    yield courant
    compte += 1
```

(3) **(sur 1,5 point)** Écrire une fonction génératrice `itere(f, x)`, qui prend en entrée une fonction `f` et un nombre `x`, et délivre successivement les valeurs

`x, f(x), f(f(x)), f(f(f(x))), ...`

tant qu'il n'y a pas deux valeurs consécutives identiques. Par exemple, si `f=lambda x:abs(x-2)` et `x=7`, alors `itere(f, x)` délivre les valeurs $7, 5, 3, 1$.

```
def itere(f, x):
  ''' function x nombre -> generator
  retourne un générateur des itérés de f sur x jusqu'à obtention d'un point fixe
  '''
  yield x
  while f(x)!=x:
    x = f(x)
    yield x
```

Exercice 6. **(au total sur 2 points)** On appelle *développement égyptien* d'un nombre rationnel toute écriture de ce nombre sous la forme d'une somme d'inverses de nombres entiers distincts. Par exemple,

$$\frac{2}{3} = \frac{1}{3} + \frac{1}{4} + \frac{1}{12}$$

est un développement égyptien du nombre rationnel $\frac{2}{3}$. Par application récursive de l'identité (que l'on ne demande pas de démontrer)

$$\frac{p}{q} = \frac{1}{q} + \frac{p-1}{q+1} + \frac{p-1}{q(q+1)},$$

on peut obtenir un développement égyptien de n'importe quel nombre rationnel strictement positif, car :

- les dénominateurs des termes de droite sont tous distincts;
- si $p > 1$, les numérateurs des termes de droite sont tous strictement inférieurs à p.

(sur 2 points) Écrire une fonction **récursive** `egyptien(p, q)` qui prend en entrée deux entiers strictement positifs `p` et `q`, et retourne une liste d'entiers correspondant aux dénominateurs d'un développement égyptien du nombre $\frac{p}{q}$. L'exemple considéré au début de l'exercice montre ainsi que `[3,4,12]` est un résultat valable pour l'appel `egyptien(2,3)`.

- Si `p=1`, la fraction $\frac{p}{q}$ est déjà sous la forme demandée ($\frac{1}{q}$), donc on retourne simplement la liste `[q]` (c'est le test d'arrêt);
- sinon, on utilise l'équation de l'énoncé, qui montre que $\frac{p}{q}$ se décompose en une somme de 3 termes: la fraction $\frac{1}{q}$ qui est sous la bonne forme, et deux autres fractions qui seront à leur tour décomposées par des appels récursifs. On renvoie donc la concaténation de `[q]` (première fraction), `egyptien(p-1,q+1)` (décomposition de la deuxième fraction) et `egyptien(p-1, q*(q+1))` (décomposition de la troisième fraction).

```
def egyptien(p,q):
  ''' int x int -> list
  retourne une liste de dénominateur réalisant une décomposition
  en fractions égyptiennes de p/q
  '''
  if p==1:
    return [q]
  return [q] + egyptien(p-1,q+1) + egyptien(p-1, q*(q+1))
```